PROJECT AND TEAM INFORMATION

Project Title

Smart Hostel Management & Complaint Resolution System

Student / Team Information

Team Name: *Team ID:*	*Hostel- Nexus* *DSCPP-III-2026-T269*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	Raj, Harshit - 2510011339 *rajharshit1807@gmail.com* 
Team member 2 *(Last Name, name: student ID: email, picture):*	Negi, Diviya - 2510010573 *diviyanegi51@gmail.com* 
Team member 3 *(Last Name, name: student ID: email, picture):*	Verma, Aditya – 2510380258 av0844769@gmail.com 

PROPOSAL DESCRIPTION

Motivation

In most college hostels, day-to-day management still depends on physical registers, notice boards, and word-of-mouth communication between students and the warden. Room and bed allotment is usually done manually at the start of a semester, which often leads to confusion when a bed is marked occupied in one register but not updated elsewhere, or when a vacant bed goes unnoticed for weeks. Complaints, whether about a broken tap, a faulty switchboard, or a mess-related issue, are generally noted down on paper or informed verbally, with no consistent way to track which ones are still pending, how long they have been pending, or which ones need urgent attention. A minor complaint like a squeaky door and a serious one like an electrical short circuit often end up waiting in the same informal queue, simply based on the order they were reported rather than their actual urgency.

Visitor entries, leave and outpass requests add another layer of manual paperwork, making it hard for a warden to quickly check who is currently inside the hostel or which students are on leave. As the number of students grows, searching for a specific student's room, checking a room's occupancy history, or locating the status of an old complaint becomes increasingly time-consuming because records are scattered across different registers and are not linked to one another.

A unified hostel management system can address these problems by keeping student, room, complaint, visitor, and leave records in one place, allowing urgent issues to be handled before routine ones, and making search and record retrieval fast and reliable. Solving this problem is important not just for convenience, but because timely handling of urgent hostel issues directly affects student safety and daily comfort.

State of the Art / Current solution

Today, hostel management in most colleges is handled through a mix of physical registers, spreadsheets, and informal communication such as WhatsApp groups or notices pinned on boards. Room and bed allocation is usually recorded manually by the warden's office, and vacancy information is updated only when someone remembers to do so, which can lead to double allocation or beds staying vacant unnoticed. Complaints and maintenance requests are typically written in a complaint register or reported verbally, and are resolved roughly in the order they arrive, without any systematic way to flag urgent issues over routine ones. Basic spreadsheet-based tools are sometimes used for student lists, but they rarely connect room data, complaint data, and visitor or leave records together. As a result, tracking a pending complaint, searching for a student's current room, or reviewing a room's past occupancy usually means manually going through multiple registers or files. This academic prototype does not claim to replace existing commercial hostel software, but focuses on demonstrating, at a college project level, how basic data structures and an object-oriented C++ design can meaningfully improve prioritization, search speed, and record-keeping over these typical manual and spreadsheet-based methods.

Project Goals and Milestones

General Goals:

- Provide a single, centralized platform to manage student records, room/bed allocation, complaints, visitors, and leave/outpass requests instead of separate registers.
- Allocate and track room and bed vacancy efficiently, avoiding double allocation.
- Record and manage hostel complaints and maintenance requests, ensuring urgent issues are handled before routine ones.
- Enable fast searching and retrieval of student, room, and complaint information as hostel occupancy grows.
- Maintain visitor entry/exit and leave/outpass records for better tracking.
- Preserve a history of room occupancy and complaint resolution for future reference.
- Demonstrate meaningful use of DSA (in C) and object-oriented C++ design in a single working system.

Milestones:

- Requirement analysis and finalizing the exact set of features and data fields.
- Design core C/C++ data structures and class hierarchy (Student, Room, Complaint, Visitor, Warden).
- Implement student registration, profile management, and room/bed allocation using hash-based lookup.
- Implement complaint and maintenance handling using a priority queue for urgent issues and a queue for routine ones.
- Implement visitor entry/exit and leave/outpass tracking, linked to student records.
- Implement searching and sorting for records (by room, urgency, or date) and maintain history using linked lists.
- Implement file handling for saving and loading records between sessions.
- Integrate all modules into a single console-based system, test with sample data, and handle edge cases such as a full hostel or an invalid room number.

Project Approach

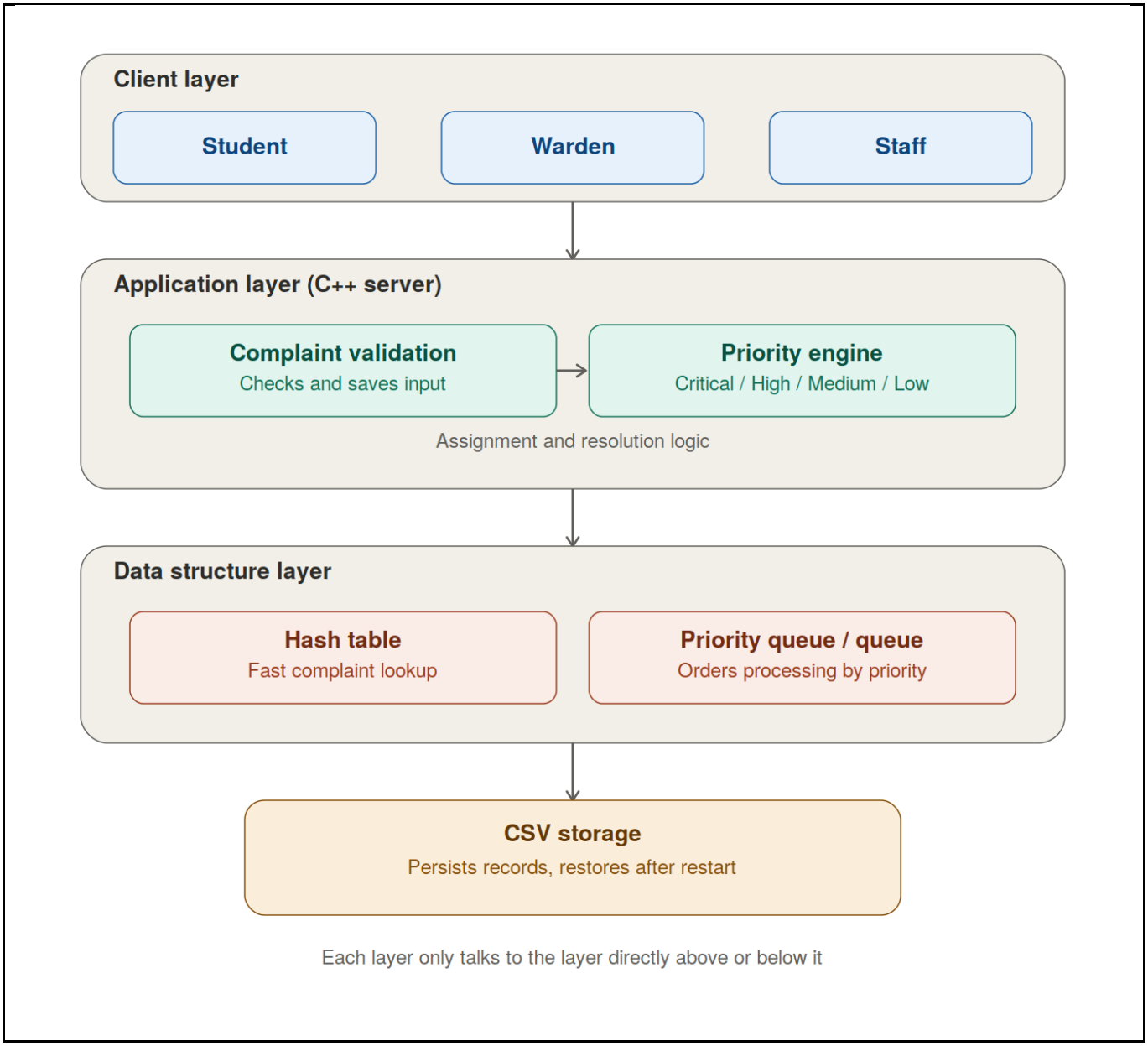
The system will be implemented as a modular, console-based application, with the core data structures and algorithms written in C to satisfy the DSA requirement, and the overall system design and application logic built in C++ using object-oriented programming.

Student, room, and complaint records will be stored using a hash table, keyed by student ID or room number, so that lookups such as "find a student's room" or "check a room's status" run in near-constant time instead of a linear scan through all records. Hostel complaints and maintenance requests will be split into two categories: urgent issues (such as electrical faults or water leakage) will be placed in a priority queue, ordered by severity so they are resolved first, while routine complaints will be handled through a normal queue on a first-come-first-served basis. Linked lists will be used to maintain the history of each room's occupancy and each student's past complaints, since these grow over time and do not need random access. Sorting algorithms will be used to

arrange complaints by date or priority and to list students or rooms in order, with searching algorithms used to look up records when hash-based lookup is not applicable.

On the C++ side, the system will use classes for Student, Room, Complaint, Visitor, and Warden, with encapsulation to protect internal data through getters/setters, and a base Person class with inheritance and a virtual display function to demonstrate polymorphism for Student and Warden. Constructors will initialise objects safely, exception handling will manage invalid inputs (such as a duplicate ID or an invalid room number), and file handling will be used to persist records between program runs. STL containers will be used only where they genuinely simplify the implementation, without replacing the core hand-built DSA components.

System Architecture (High Level Diagram)



Project Outcome / Deliverables

The final deliverable will be a working C++ console-based prototype of the Smart Hostel Management System, demonstrating the following functionality:

- Student registration and profile management, with fast lookup by student ID or name.
- Room and bed allocation with real-time vacancy tracking to avoid double allocation.
- Complaint and maintenance request logging, with urgent issues handled first through a priority queue and

routine ones through a normal queue.

- Visitor entry/exit recording and leave/outpass tracking linked to student records.
- Searching and sorting of student, room, and complaint data by relevant criteria such as urgency or date.
- File-based storage so that records persist between program runs.
- Maintenance of room occupancy history and complaint history using linked lists.
- A demonstration run using sample data, including edge cases such as a full hostel, an invalid room number, or a duplicate student ID.

The prototype will clearly show C being used for the underlying data structures and algorithms, and C++ being used for the object-oriented application design, matching the goals and approach described above.

Assumptions

We assume a single hostel (or a small, fixed number of blocks) with a predefined number of rooms and beds. Student, complaint, and visitor data is sample/mock data entered for demonstration, not pulled from a live college database. There is no real-time online database, biometric hardware, GPS/live location tracking, or payment gateway involved. The system is a console-based prototype intended to demonstrate DSA and C++ concepts rather than a deployment-ready application, and urgency levels for complaints are assigned manually rather than through automated sensors.

References

- cppreference.com · C++ language and standard library reference (classes, inheritance, polymorphism, exception handling, STL): https://en.cppreference.com/
- GeeksforGeeks · Data Structures and Algorithms: Hash Table, Priority Queue, Queue, Linked List, Searching and Sorting algorithms: https://www.geeksforgeeks.org/data-structures/
- GeeksforGeeks · File Handling in C/C++: https://www.geeksforgeeks.org/file-handling-c-classes/
- Design and implementation of automated secure web-based hostel management system with automated room allocation and complaint tracking: https://www.researchgate.net/publication /401357924_Design_and_implementation_of_automated_secure_web-based_hostel _management_system_with_automated_room_allocation_and_complaint_tracking
- Development of Web-Based Hostel Management System: https://www.researchgate.net /publication/389336918_Development_of_Web-Based_Hostel_Management_System
- Hybrid Technology Based Smart Hostel Management System Using Artificial Intelligence and Internet of Things: https://www.researchgate.net/publication/369283563_Hybrid_Technology _Based_Smart_Hostel_Management_System_Using_Artificial_Intelligence_and_Internet_of_Things